Part 1 – Pattern discovery: Member 1 prompt and result

Course 23CSE455 Design Patterns, Team A15
Member: Hariprid V (AM.AI.U4AID23009)
Scope: presentation / contract layer (rest/)
Model: Claude Sonnet 5.5 (configured id claude-sonnet-5-5), 2026-10-11
Attached file: `input_source_numbered.txt` (numbered Java source of the baseline application, 20 KB; it is in `Part1/llm/`).

Prompt

```
[Member 1 prompt - Part 1]
SCOPE: presentation and contract layer = rest/ (AccountController, UserController),
persistence/dto/ (AccountDTO, UserDTO, TransactionDTO) and exception/ (BadRequestException, NotFoundException).
You may name classes outside the scope when they are collaborators, but only report candidates whose
main participants are in this scope.

You are assisting with a university Design Patterns case study (course 23CSE455).

INPUT: the numbered Java source of a Quarkus bank REST API (baseline version), supplied as
input_source_numbered.txt. Every source line starts with its real line number followed by '|'.

TASK (Part 1, pattern discovery) - restricted to the scope given below:
1. Find every pattern candidate in your scope: GoF patterns, GRASP patterns, enterprise/architecture
   patterns, and framework idioms. Give the exact recognised pattern name and its category.
2. For each candidate give a meaningful instance name (one collaboration, e.g. "Book Return Notification",
   not just "Strategy").
3. Map every pattern role to the ACTUAL class / interface / method in the code. Do not invent classes.
   If a role is combined or absent, say "not separately declared".
4. Give a file path and line range for every participant, using this exact row format:
       ROLE | SYMBOL | path:start-end
   where path is relative to src/main/java/br/com/michaelcruz/ (or src/main/resources/).
5. Give concise evidence from the code (relationships, calls, delegation, behaviour).
   A class name or an annotation alone is NOT sufficient evidence.
6. Give a verdict for each candidate: CONFIRMED, REJECTED or UNCERTAIN, with a reason.
   Two concrete classes in one collaboration are ONE instance.
7. Finish with the counts for your scope: confirmed pattern types, confirmed instances,
   concrete participant classes, rejected candidates, uncertain candidates.
Do not use any knowledge about this repository other than the supplied code.
```

Result

```
# Member 1 - raw model output (scope: rest/, persistence/dto/, exception/)

## Candidate 1 - "REST Use-Case Controller" - GRASP Controller - CONFIRMED
Problem solved: HTTP requests (system events) must be received and handed to the business layer without the
HTTP layer containing business rules.

Controller | AccountController | rest/AccountController.java:13-50
Controller | UserController | rest/UserController.java:10-21
SystemOperation | AccountController.createAccount | rest/AccountController.java:21-26
SystemOperation | AccountController.depositToAccount | rest/AccountController.java:28-34
SystemOperation | AccountController.withdraw | rest/AccountController.java:36-42
SystemOperation | AccountController.listAccounts | rest/AccountController.java:44-48
SystemOperation | UserController.createUser | rest/UserController.java:16-20
Delegate | AccountService (injected field accountService) | rest/AccountController.java:16-17
Delegate | UserService (injected field userService) | rest/UserController.java:13-14

Evidence: every controller method unwraps the DTO (e.g. depositDTO.getAmount() at AccountController line 31),
makes exactly one call to a service (lines 23, 32, 40, 46; UserController line 18) and wraps the outcome in a
Response (lines 25, 33, 41, 47; UserController line 19). No balance, validation or persistence logic appears in
the controllers. Neither controller touches a DAO.

## Candidate 2 - "API Payload DTO" - Data Transfer Object - CONFIRMED (partial)
Problem solved: decouple the JSON request contract from the persistence entities.

DataTransferObject | UserDTO | persistence/dto/UserDTO.java:5-60
DataTransferObject | AccountDTO | persistence/dto/AccountDTO.java:6-50
DataTransferObject | TransactionDTO | persistence/dto/TransactionDTO.java:5-15
Consumer | AccountController.createAccount | rest/AccountController.java:21-26
Consumer | AccountController.depositToAccount | rest/AccountController.java:28-34
```

Consumer | AccountController.withdraw | rest/AccountController.java:36-42
Consumer | UserController.createUser | rest/UserController.java:16-20
Assembler | not in my scope (see util/UserUtil and util/AccountUtil, Member 2)

Evidence: the three DTO classes contain only fields, bean-validation annotations and accessors (no behaviour) and are the declared parameter types of the controller methods. The mapping to entities happens outside the controllers. It is only partial because listAccounts (lines 45-47) returns the Account entities themselves, so the output direction is not protected by a DTO.

Candidate 3 - "HTTP Error Translation" - Exception Translation (JAX-RS framework idiom) - CONFIRMED
Problem solved: business failures must become HTTP 400 / 404 responses with a message body.

TranslatedException | BadRequestException | exception/BadRequestException.java:6-11
TranslatedException | NotFoundException | exception/NotFoundException.java:6-11
Translator | not separately declared (the JAX-RS runtime turns WebApplicationException.getResponse() into the HTTP reply)

Evidence: both classes extend jakarta.ws.rs.WebApplicationException and build the Response with the status (BAD_REQUEST / NOT_FOUND) and the message as entity in their constructor (lines 8-10). They are thrown from service/ and dao/ code that I did not analyse in detail.

Rejected / uncertain
- Builder (Response.status(..).entity(..).build()) - REJECTED as an application instance: the builder is implemented by JAX-RS (dependency); the controllers are only clients of it.
- Front Controller - REJECTED: there is no dispatcher class in the supplied code; dispatching is done by the framework.

Counts for my scope
confirmed pattern types: 3; confirmed instances: 3; concrete participant classes: 7 (2 controllers, 3 DTOs, 2 exceptions); rejected: 2; uncertain: 0.
Side observation (not a pattern): AccountController line 24 contains System.out.println(accountDTO).